Notes

Gabriele Genovese

23 April 2025

Contents

1 A Partial Order View of Message-Passing Communication Models POPL23 1
1.1 Definition of Message Sequence Chart (MSC) ... 2
1.2 MSC Classes ... 2
1.2.1 Fully async ... 2
1.2.2 Peer-to-peer ... 2
1.2.3 Causally ordered ... 3
1.2.4 Mailbox ... 3
1.2.5 FIFO 1-n ... 3
1.2.6 FIFO n-n ... 3
1.2.7 RSC ... 3

2 Fossacs 2020 (Leatitia) 4
2.1 Definition of conflict graph ... 4
2.2 Definition of k-synchronous ... 4

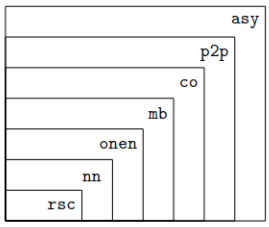
3 Loic thesis 4
3.1 Definition of Action Graph ... 4
3.2 Definition of borderline violation ... 4

4 Paper Alur (Realizability) 4
4.1 Verification (probabilemente non mi serve) ... 5

1 A Partial Order View of Message-Passing Communication Models POPL23

$\mathbb{P}$: set of processes/partecipants $(p, q, r, ...)$

$\mathbb{M}$: set of messags $(m1, m2, m3, ...)$



(a) The hierarchy of MSC classes.

	Weakly sync	Weakly k-sync	∃k bounded	∀k bounded
asy	unbounded STW	✓	✓	✓
p2p	✗ [1]	✓ [1]	✓ [1]	✓ [1]
co	✗	✓	✓	✓
mb	✓ [1]	✓ [1]	✓ [1]	✓ [1]
onen	✓	✓	✓	✓
nn	✓	✓	✓	✓

(b) (Un)decidability results for the synchronizability problems, [1] indicates that the result was shown in [3].

Figure 2: Main contributions

1.1 Definition of Message Sequence Chart (MSC)

An MSC over $\mathbb{P}$ and $\mathbb{M}$ is a tuple $M = (\mathrm{E}, \rightarrow, \lhd, \lambda)$ where E is finite (possibly empty) set of events, $\lambda : \mathrm{E} \rightarrow \Sigma$ is a labelling function that associates an action to each event, and $\rightarrow, \lhd$ are binart relations on E that satisfy the following three conditions. For $p \in \mathbb{P}$, let $\mathrm{E}_p = \{e \in \mathrm{E} \mid \lambda(e) \in \Sigma_p\}$ be the set of vents that are executed by p.

1. The process relation $\rightarrow \subseteq \mathrm{E} \times \mathrm{E}$ relates an event to its immediate successor on the same process: $\rightarrow = \bigcup_{p \in \mathbb{P}} \underset{p}{\rightarrow}$ for some relations $\underset{p}{\rightarrow} \subseteq \mathrm{E}_p \times \mathrm{E}_p$ such that $\underset{p}{\rightarrow}$ is the direct successor relation of a total order on E_p.
2. The message relation $\lhd \subseteq \mathrm{E} \times \mathrm{E}$ relates pair of matching send/receive events:

   2a. for every pair $(e, f) \in \lhd$, there are two processes p, q and a message m such that $\lambda(f) = \mathrm{send}(p, q, m)$ and $\lambda(f) = \mathrm{rec}(p, q, m)$.

   2b. for all $f \in \mathrm{E}$, with $\lambda(f) = \mathrm{rec}(p, q, m)$, there is exactly one $e \in \mathrm{E}$ such that $e \lhd \mathrm{E}$.

   2c. for all $e \in \mathrm{E}$ such that $\lambda(f) = \mathrm{send}(p, q, m)$, there is at most one $f \in \mathrm{E}$ such that $e \lhd f$.
3. The happens-before relation $\underset{\mathrm{hb}}{\leq}$, defined by $(\rightarrow \cup \lhd)^*$, is a partial order on E.

1.2 MSC Classes

1.2.1 Fully async

In the fully asynchronous communication model (`asy`), messages can be received at any time once they have been sent, and send events are non-blocking. It can be modeled as a bag where all messages are stored and retrieved by processes when necessary. It is also referred to as NON-FIFO. The definition is the same as MSC.

1.2.2 Peer-to-peer

In the peer-to-peer (`p2p`) communication model, any two messages sent from one process to another are always received in the same order as they are sent. This is usually implemented by processes pairwise connected with FIFO channels. Alternative names are FIFO 1−1 or simply FIFO.

A p2p-MSC is an MSC $M = (E, \rightarrow, \lhd, \lambda)$ where, for any two send events s and s' such that $\lambda(s) \in \mathrm{send}(p, q, _), \lambda(s') \in \mathrm{send}(p, q, _)$, and $s \overset{+}{\rightarrow} s'$, one of the following holds:

- either $s, s' \in \mathrm{matched}(M)$ with $s \lhd r$ and $s' \lhd r'$ and $r \overset{+}{\rightarrow} r'$,
- or $s' \in \mathrm{unmatched}(M)$.

Note that we cannot have two messages $m1$ and $m2$, both sent by p to q, in that order, such that $m1$ is unmatched and $m2$ is matched; unmatched message $m1$ excludes the reception of any later message.

1.2.3 Causally ordered

In the causally ordered (`co`) communication model, messages are delivered to a process according to the causality of their emissions. In other words, if there are two messages $m1$ and $m2$ with the same recipient, such that there exists a causal path from $m1$ to $m2$, then $m1$ must be received before $m2$. Causal ordering was introduced by Lamport with the name "happened before" order.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is causally ordered if, for any two send s and s', such that $\lambda(s) \in \text{send}(_, q, _), \lambda(s') \in \text{send}(_, q, _)$, and $s \underset{\text{hb}}{\leq} s'$:

- either $s, s' \in \text{matched}(M)$ and $r \overset{*}{\rightarrow} r'$, with r and r' receive events such that $s \lhd r$ and $s' \lhd r'$.
- or $s' \in \text{unmatched}(M)$.

Note that in a `co`-MSC we cannot have two send events s and s' addressed to the same process, such that s is unmatched, s' is matched, and $s \underset{\text{hb}}{\leq} s'$.

1.2.4 Mailbox

In this model, any two messages sent to the same process, regardless of the sender, must be received in the same order as they are sent. If a process receives m_1 before m_2, then m_1 must have been sent before m_2. `mb` coordinates all the senders of a single receiver. This model is also called FIFO $n-1$.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is a `mb`-MSC if it has a linearization $\rightsquigarrow$ where, for any two send events s and s', such that $\lambda(s) \in \text{send}(_, q, _), \lambda(s') \in \text{send}(_, q, _)$, and $s \rightsquigarrow s'$

- either $s, s' \in \text{matched}(M)$ and $r \rightsquigarrow r'$, where $s \lhd r$ and $s' \lhd r'$,
- or $s' \in \text{unmatched}(M)$.

1.2.5 FIFO 1-n

This model is the dual of `mb`, it coordinates a sender with all the receivers. Any two messages sent by a process must be received in the same order as they are sent. These two messages might be received by different processes and the two receive events might be concurrent.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is a `onen`-MSC if it has a linearization $\rightsquigarrow$ where, for any two send events s and s', such that $\lambda(s) \in \text{send}(p, _, _), \lambda(s') \in \text{send}(p, _, _)$ and $s \overset{+}{\rightarrow} s'$ (which implies $s \rightsquigarrow s'$)

- either $s, s' \in \text{matched}(M)$ and $r \rightsquigarrow r'$, with r and r' receive events such that $s \lhd r$ and $s' \lhd r'$,
- or $s' \in \text{unmatched}(M)$.

1.2.6 FIFO n-n

In this model, messages are globally ordered and delivered according to their emission order. Any two messages must be received in the same order as they are sent. These two messages might be sent or receives by any process and the two send or receive events might be concurrent. The FIFO `n-n` coordinates all the senders with all the receivers.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is a `nn`-MSC if it has a linearization $\rightsquigarrow$ where, for any two send events s and s', such that $s \rightsquigarrow s'$

- either $s, s' \in \text{matched}(M)$ and $r \rightsquigarrow r'$, with r and r' receive events such that $s \lhd r$ and $s' \lhd r'$,
- or $s' \in \text{unmatched}(M)$.

1.2.7 RSC

This model imposes the existence of a scheduling such that any send event is immediately followed by its corresponding receive event.

An MSC $M = (E, \rightarrow, \lhd, \lambda)$ is an `rsc`-MSC if it has no unmatched send events and there is a linearization $\rightsquigarrow$ where any matched send event is immediately followed by its respective receive event.

2 Fossacs 2020 (Leatitia)

2.1 Definition of conflict graph

Intuitively, we have a dependency whenever two messages have a process in common.

The conflict graph `CG(e)` of a sequence of action $e = a_1 ... a_n$ is the labeled graph $\left(V, \{\overset{XY}{\rightarrow}\}_{X,Y \in \{R,S\}}\right)$ where V is the set of message exchanges of e, and for all $X, Y \in \{S, R\}$, for all $v, v' \in V$, there is a XY dependency edge $v \overset{XY}{\rightarrow} v'$ between v and v' if there are $i < j$ such that $\{a_i\} = v \cap X, \{a_j\} = v' \cap Y$, and $\text{proc}_{X(v)} = \text{proc}_{Y(v')}$.

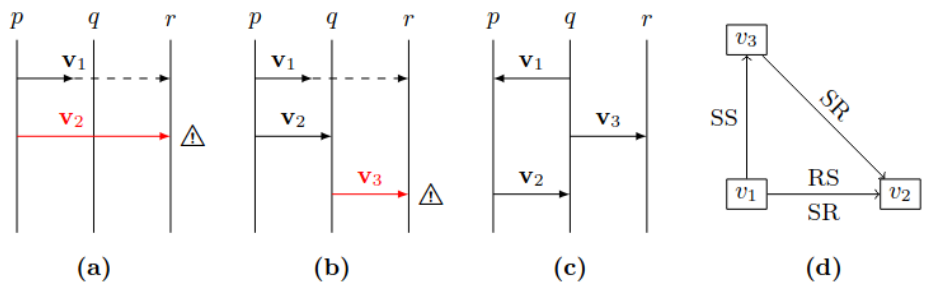


Fig. 1: (a) and (b): two MSCs that violate causal delivery. (c) and (d): an MSC and its conflict graph

2.2 Definition of k-synchronous

k denotes a given integer $k \geq 1$. A k-exchange denotes a sequence of action starting with at most k sends and followed by at most k receives matching some of the sends. An MSC is k-synchronous if there exists a linearisation that is breakable into a sequence of k-exchanges sucj that a message sent during a k-exchange cannot be received during a subsequent one: either it is received during the same k-exchange, or it remains orphan forever.

An MSC `msc` is k-synchronous if:
1. there exists a linearisation of m $e = e_1 \cdot e_2 ... e_n$ where for all $i \in [1...n]$, $e_i \in S^{\leq k} \cdot R^{\leq k}$,
2. m satisfies causal delivery,
3. for all j, j' such that $a_j \mid - \mid a_{j'}$ holds in $e, a_j \mid - \mid a_{j'}$ holds in some e_i.

An execution e is k-synchronizable if $m(e)$ is k-synchronous.

We write $sTr_k(\mathfrak{S})$ to denote the set $\{m(e) \mid e \in asEx(\mathfrak{S})$ and $m(e)$ is k-synchronous$\}$.

3 Loic thesis

3.1 Definition of Action Graph

Let $\bigwedge_{\mathbb{P},\mathbb{I},\mathbb{V}}$ be an alphabet of actions, and $e = a_1, ..., a_n$ be an execution over this alphabet. The action graph of e, denoted $\texttt{agraph}(e)$, is the vertex-labelled directed graph $(\{1, ..., n\}, \underset{e}{\prec}, \lambda)$, where for all $i \in \{1, ..., n\}, \lambda(i) = a_i$, and arcs follow from causal dependency: there is an arc between vertices i and j if $i \underset{e}{\prec} j$.

3.2 Definition of borderline violation

Let $\bigwedge_{\mathbb{P},\mathbb{I},\mathbb{V}}$ be an alphabet of communications. A borderline violation is an execution $e = e' \cdot \iota ?^p \mathrm{v}$ over $\bigwedge_{\mathbb{P},\mathbb{I},\mathbb{V}}$ such that e' is RSC, and e is not causally equivalent to an RSC execution.

4 Paper Alur (Realizability)

Theorem 1: Given a bounded MSC graph G, checking if G is *weakly realizable* is **undecidable**. **Proof**: The proof is a reduction from the post correspondence problem (PCP).

Lemma 2: R(elaxed)PCP is undecidable.

Theorem 3: Checking *safe realizability* of a bounded MSC-graph is in `EXPSPACE`.

Theorem 4: Checking *safe realizability* of a bounded MSC-graph is `PSPACE`-hard.

4.1 Verification (probabilemente non mi serve)

Theorem 5: Given an MSC-graph G and an MSC M over k processes, there is an algorithm that decides in $O(|G|\ |M|\ k)$ time whether $M \in L(G)$.

Theorem 6: Given an MSC-graph G and an MSC M, it is `NP`-complete to determine if $M \in L(G)$, even when G is a complete graph, or when G is an acyclic graph.

Theorem 7: Given an MSC-graph G and an MSC M, there is an algorithm that in time $O(|G|\ |M|)$ determines whether $M \in L^w(G)$.

Theorem 8: There are local properties φ_1 and φ_2 such that for a finite MSC set L, it is `coNP`-complete to determine if every MSC in L^w satisfies $\varphi_1 \vee \varphi_2$.

Theorem 9: There is a boolean combination φ of local properties, such that given a bounded MSC-graph G, it is undecidable to check if every MSC in $L^w(G)$ satisfies φ.